

Deep Learning for Beginners

A Practical TensorFlow Program

A Beginner's Companion Guide

How TensorFlow trains a neural network for you — variables, gradient tape, and the building blocks behind the code

About This Guide

From NumPy by Hand to a Framework That Does the Calculus for You

A plain-language walkthrough of a working TensorFlow program — and the ideas behind every line

Up to now you have built neural networks more or less from scratch in Python and NumPy, computing every gradient by hand. That was the right way to learn, because it forced you to understand what these algorithms really do. But as models grow — convolutional networks, recurrent networks, very large architectures — writing all of the backpropagation yourself stops being practical. The same way you would not write your own matrix-multiply routine for a large application, you would not hand-derive every gradient. You reach instead for a framework.

This guide walks through a single, runnable TensorFlow program, `tensorflow_demo.py`, that ties together every idea from the Week 3 assignment. It starts with the smallest possible example — minimizing one cost function — and builds up to a full forward pass and a real loss. Each section explains a concept in everyday language, shows the code that implements it, and notes where the idea fits in the wider world of deep learning.

HOW TO READ THIS GUIDE

Each section follows the same rhythm: the concept in plain terms, the code that implements it, and a short note on why it matters. You need no mathematics beyond multiplication and the idea of a slope. The questions the assignment raises — what a Variable is, how the gradient gets computed for you, why softmax is skipped until the very end — are answered directly where they arise, and gathered again at the close.

What Is TensorFlow, in Plain Terms

Imagine you are standing at the bottom of a valley in thick fog, trying to find the very lowest point — but you cannot see anything. What you *can* do is feel the ground under your feet. If it slopes downhill to your left, you take a small step left. You feel again, step again, and keep going. Step by step, you end up at the bottom of the valley — without ever seeing it — just by always stepping in the downhill direction.

That is essentially what TensorFlow does to train a model. The “valley” is the model's **error** — how wrong its guesses are. The “lowest point” is the set of values that makes the model least wrong. TensorFlow keeps nudging those values downhill until the error is as small as it can get.

The clever part is the “feeling the slope” step. Working out which way is downhill normally takes a lot of fiddly calculus. **TensorFlow does that math for you automatically.** You only have to tell it one thing — how to measure how wrong you are right now — and it works out the downhill direction and takes the steps on its own.

Why a Mathematical “Finding” — and How the Demo Makes It Concrete

Training is really a search for the best numbers, and “best” is defined mathematically: the values that make the error smallest. So finding the answer means finding the bottom of that error valley. The demo script shows this with the smallest possible example. You hand TensorFlow a single rule:

My error is: $w^2 - 10w + 25$

You never tell it the answer. You just press “go” a thousand times, and each time it feels the slope and steps downhill. After a thousand steps it lands on $w = 5$ — which happens to be the exact bottom of that valley. It found the answer entirely by itself, purely from your description of the error. That is the whole idea in miniature: *you describe the goal, and TensorFlow does the calculus and the step-by-step searching to reach it.*

Why TensorFlow Is Needed in Deep Learning

In the tiny example there is just one number to find, w , so you could almost solve it by hand. A real deep network is the same idea blown up to an enormous scale: instead of one number it has millions of weights, all tangled together across many layers, and the error valley is a landscape in millions of dimensions that no one can picture. Three things make doing this by hand impossible — and are exactly what TensorFlow handles for you.

WHY THE FRAMEWORK IS NEEDED

First, the calculus. Finding the downhill direction for millions of weights means millions of linked derivatives (backpropagation) — easy to get subtly wrong by hand, but TensorFlow derives them all automatically from the forward pass alone. Second, the speed. Each step recomputes the whole network over large datasets; TensorFlow runs this on optimized hardware (CPU/GPU) far faster than plain Python. Third, the flexibility. Swapping an optimizer, adding a layer, or changing the model is often a one-line change instead of re-deriving everything. You learned backprop by hand in NumPy so you understand what is happening — TensorFlow then lets you do it at a scale where by-hand simply is not practical.

1. The One Idea That Makes TensorFlow Worth It

There is a single observation at the heart of every framework. When you train a network, you write two kinds of code: **forward propagation**, which computes the cost J , and **backward propagation**, which computes the gradients used to improve the weights. Forward prop is easy. Backward prop — all those chained derivatives — is where the bugs live.

TensorFlow's promise is that you only have to write the forward step. You specify how to compute the cost, and the framework figures out the derivatives and the gradients automatically. That is the whole reason a framework can make you more efficient: it removes the hardest, most error-prone part of the work.

KEY CONCEPT: YOU ONLY WRITE FORWARD PROP

In TensorFlow you implement the cost function — the forward pass — and nothing else about the gradients. The framework records the operations you performed and replays them backwards to produce every derivative for you. Swapping in a different optimizer, or changing the model, is then often a one-line change.

2. Variables and Constants: What Can Change, and What Cannot

Before anything can be trained, TensorFlow needs to know which numbers are allowed to move. That is the difference between a **tf.Variable** and a **tf.constant**. A Variable holds state you intend to change — the weights and biases the optimizer will adjust. A constant holds state you will not change — your fixed input data. You can modify a Variable; you cannot modify a constant.

```
w = tf.Variable(0.0, dtype=tf.float32)    # trainable: state can change
x = tf.constant([1.0, -10.0, 25.0])      # fixed data: state cannot change
```

This single distinction is what lets the framework know what to optimize. When you later ask for gradients, TensorFlow differentiates with respect to the Variables and leaves the constants alone.

KEY CONCEPT: VARIABLE vs CONSTANT

Use `tf.Variable` for anything the network learns (weights, biases) — its value is updated every training step. Use `tf.constant` for fixed inputs and data. Declaring `W` as a Variable is precisely how you tell TensorFlow, “this is the thing I want you to optimize.”

3. GradientTape: A Cassette Tape for Calculus

How does TensorFlow take derivatives of code it has never seen? It records them. The construct **tf.GradientTape** watches every operation you perform on your Variables while computing the cost, writing them down in order — like an old cassette tape recording the forward pass. When you play that tape backwards, it can revisit each operation in reverse and compute the gradient along the way. That reverse replay is backpropagation, done for you.

The motivating problem from the lecture is deliberately tiny: minimize the cost $J(w) = w^2 - 10w + 25$. That factors as $(w - 5)^2$, so the minimum is at $w = 5$ — but we pretend we do not know that and let TensorFlow find it. We supply the data as a small array $x = [1, -10, 25]$ that plays the role of the cost's coefficients:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.1)

def cost():
    # x0*w^2 + x1*w + x2  ->  w^2 - 10w + 25
    return x[0] * w ** 2 + x[1] * w + x[2]

def train_step():
    with tf.GradientTape() as tape:          # record the forward pass
        J = cost()
        trainable = [w]                    # variables to optimize
        grads = tape.gradient(J, trainable) # replay backwards -> gradients
        optimizer.apply_gradients(zip(grads, trainable))
```

Run that step a thousand times and W climbs from its starting value of 0 to almost exactly 5 — the true minimum — without us ever writing a derivative. The Python **zip** simply pairs each gradient with the variable it belongs to so the optimizer knows what to update.

KEY CONCEPT: THE GRADIENT TAPE

`tf.GradientTape` records the sequence of operations in the forward pass. Calling `tape.gradient(J, variables)` plays that recording backwards to compute every gradient automatically. You provide the cost; the tape provides the calculus. This is why declaring W as a `Variable` and computing the cost inside the tape's scope is all the setup backprop needs.

4. The Building Blocks: Linear, Sigmoid, One-Hot

With the core loop understood, the assignment's smaller exercises are each a single, sharp idea expressed in the TensorFlow API. The **linear function** is the workhorse of every layer: a matrix multiply plus a bias, $Y = WX + b$. Note that the data tensors are constants, not `Variables`, because here we are only computing — not training:

```
def linear_function():
    np.random.seed(1)
    X = tf.constant(np.random.randn(3, 1), name="X")
    W = tf.constant(np.random.randn(4, 3), name="W")
    b = tf.constant(np.random.randn(4, 1), name="b")
    return tf.add(tf.matmul(W, X), b)      # matrix product + bias
```

The **sigmoid** squashes any real number into the range (0, 1). The only subtlety is `dtype`: the Keras activation needs a floating-point input, so we cast first. The **one-hot** encoder turns a single integer class label into a column of zeros with a single 1, then reshapes it to the length you asked for:

```
def sigmoid(z):
    z = tf.cast(z, tf.float32)           # activation needs a float
    return tf.keras.activations.sigmoid(z)

def one_hot_matrix(label, C=6):
    return tf.reshape(tf.one_hot(label, C, axis=0), shape=[C, ])
```

KEY CONCEPT: USE THE TF API, NOT NUMPY

Each block above uses TensorFlow operations — `tf.matmul`, `tf.add`, `tf.cast`, `tf.one_hot`, `tf.reshape` — rather than NumPy. This matters because only TensorFlow operations are recorded by the gradient tape. An equivalent NumPy line would compute the right number but would be invisible to `autodiff`, breaking training.

5. Initializing a Network and Running It Forward

A real layer's weights cannot all start at zero or at the same value, or every neuron would learn the same thing. The **Glorot (Xavier) normal** initializer draws starting weights from a distribution scaled to the layer's size,

which keeps signals from vanishing or exploding as they pass through the network. Each weight matrix and bias is a Variable, because these are exactly the numbers the optimizer will train:

```
def initialize_parameters(layer_dims):
    initializer = tf.keras.initializers.GlorotNormal(seed=1)
    params = {}
    for l in range(1, len(layer_dims)):
        params[f"W{l}"] = tf.Variable(
            initializer(shape=(layer_dims[l], layer_dims[l-1])))
        params[f"b{l}"] = tf.Variable(
            initializer(shape=(layer_dims[l], 1)))
    return params
```

Forward propagation then chains the layers: **LINEAR** → **RELU** → **LINEAR** → **RELU** → **LINEAR**. The ReLU activation keeps the positive part of each value and zeroes the rest, giving the network its non-linearity. Notice that the final layer stops at the raw scores — the *logits* — and deliberately does **not** apply softmax:

```
def forward_propagation(X, parameters):
    w1, b1 = parameters["w1"], parameters["b1"]
    w2, b2 = parameters["w2"], parameters["b2"]
    w3, b3 = parameters["w3"], parameters["b3"]
    Z1 = tf.math.add(tf.linalg.matmul(w1, X), b1)
    A1 = tf.keras.activations.relu(Z1)
    Z2 = tf.math.add(tf.linalg.matmul(w2, A1), b2)
    A2 = tf.keras.activations.relu(Z2)
    Z3 = tf.math.add(tf.linalg.matmul(w3, A2), b3)    # logits, no softmax
    return Z3
```

KEY CONCEPT: WHY STOP BEFORE SOFTMAX

Softmax turns logits into probabilities, but it is numerically safer to let the loss function apply it internally. So forward propagation returns raw logits, and the loss is told to do the softmax itself. Leaving softmax out here is intentional — it is not a missing step.

6. The Loss: Where Softmax Finally Happens

The total loss measures how wrong the predictions are. We use **categorical cross-entropy**, and the key parameter is **from_logits=True**: this tells the loss to apply softmax internally before measuring the error, which is exactly why the forward pass left it out. The labels and logits are transposed because the loss expects the shape (examples, classes), and **tf.reduce_sum** adds up the per-example losses across the batch:

```
def compute_total_loss(logits, labels):
    return tf.reduce_sum(
        tf.keras.losses.categorical_crossentropy(
            tf.transpose(labels), tf.transpose(logits),
            from_logits=True))
```

KEY CONCEPT: `from_logits=True`

Passing `from_logits=True` lets the loss take raw scores and apply softmax in one numerically stable step, rather than you applying softmax and then taking its log separately. Pair it with a forward pass that returns logits, and the two halves fit together exactly.

7. The Whole Program, Running

Put the pieces in order and the script runs top to bottom, printing each result. The output below is the actual run — note how W reaches 5, the sigmoid and one-hot values match expectation, the forward pass produces a tidy matrix of logits, and the loss collapses to a single number:

```
PART 1 -- Variables vs constants
W starts as: 0.0

PART 2 -- Minimizing a cost function
After 1000 steps, W is approximately: 5.0

PART 3 -- Linear function, sigmoid, one-hot
sigmoid(0) = 0.5
sigmoid(-1) = 0.268941
one_hot(label=2, C=5) = [0. 0. 1. 0. 0.]

PART 4 -- forward pass logits (3 classes x 2 examples):
[[ 0.452  0.397]
 [-0.538 -0.592]
 [ 1.411  1.237]]

PART 5 -- total loss over the batch: 3.675418
```

Every concept from the assignment has now appeared in one coherent program: Variables and constants, a gradient tape, an optimizer, the linear and activation building blocks, Glorot initialization, a forward pass, and a loss that supplies its own softmax.

8. The Main Takeaways

TensorFlow earns its place by removing the hardest part of training: you write only the forward computation of the cost, and the framework computes every gradient for you. A **`tf.Variable`** marks what is learnable; a **`tf.constant`** marks what is fixed. A **`GradientTape`** records the forward pass so it can be replayed backwards into gradients, which an **`optimizer`** like Adam applies. The building blocks — linear, sigmoid, one-hot, ReLU — are all expressed through the TensorFlow API so autodiff can see them, and the network's forward pass returns logits so the **cross-entropy loss** can apply softmax itself with `from_logits=True`.

The deepest lesson echoes the one from gradient checking: understanding the machinery by hand is what lets you trust the framework that hides it. You learned backprop in NumPy so that when TensorFlow does it for you in a single line, you know exactly what that line is doing — and why it works.